



Tecnológico de Monterrey

INSTITUTO TECNOLÓGICO Y DE ESTUDIOS SUPERIORES DE MONTERREY

CAMPUS PUEBLA

Análisis y diseño de algoritmos avanzados

El paradigma de diseño de algoritmos "divide y vencerás"

ATP Analyzer 2025

Fecha:

21 de Septiembre de 2026

Estudiantes:

Juan Pablo Carrera Martínez - A01739754

Diego Sanchez Lopez - A01739238

Percentil ATP: partición de Lomuto y quickselect aplicados a estadísticas de tenis

Ejercicio 2 — Algoritmos

Juan Pablo Carrera Martinez · Diego Sanchez Lopez

1. Descripción general del problema y de los algoritmos implementados

El ejercicio pide implementar una miniaplicación de la vida real que utilice el algoritmo de partición de Lomuto, con una interfaz de usuario distinta de la línea de comandos. La aplicación desarrollada, "Percentil ATP", resuelve el siguiente problema: dado un conjunto de estadísticas reales de los 50 tenistas mejor rankeados del ATP (edad, puntos de ranking, altura, aces promedio por partido y porcentaje de primer saque adentro, obtenidos de los CSV públicos de Jeff Sackmann bajo licencia CC BY-NC-SA 4.0), el usuario elige una estadística y un percentil (0-100) mediante un control deslizante (slider). La aplicación calcula el valor de corte correspondiente a ese percentil y marca visualmente qué jugadores quedan dentro y cuáles fuera del percentil elegido. Adicionalmente, el usuario puede introducir un valor propio y ver en qué percentil quedaría respecto al conjunto de datos.

La aplicación está construida como un servicio web: un backend en FastAPI (Python) que expone el cálculo del percentil, y un frontend estático en HTML/CSS/JavaScript (sin frameworks ni herramientas de build) que provee la interfaz gráfica. Esto satisface el requisito de "interfaz de usuario sencilla, no línea de comandos".

El problema de fondo, calcular el elemento correspondiente a un percentil dado, es un caso particular del problema de selección de estadísticos de orden: encontrar el k -ésimo elemento más pequeño de un arreglo sin necesidad de ordenarlo por completo. Para resolverlo se implementó el algoritmo quickselect, que se apoya en la partición de Lomuto como subrutina para reordenar el arreglo alrededor de un pivote en una sola pasada. Se implementaron dos variantes de quickselect que comparten la misma rutina de partición pero difieren en cómo eligen el pivote:

- quickselect con pivote ingenuo (quickselect_naive): usa siempre el último elemento del segmento como pivote, exactamente como lo hace la partición de Lomuto clásica.
- quickselect con selección de pivote por mediana de medianas (quickselect_mom): implementa el algoritmo de Blum, Floyd, Pratt, Rivest y Tarjan (1973), que divide el arreglo en grupos de 5 elementos, calcula la mediana de cada grupo y recursivamente encuentra la mediana de esas medianas para usarla como pivote, garantizando particiones más balanceadas.

Ambas variantes reutilizan la misma función `lomuto_partition`; la diferencia está en cómo se determina el valor del pivote antes de particionar. La corrección de las tres rutinas (partición, quickselect ingenuo y quickselect con mediana de medianas) se validó con `src/test_correctness.py`, que ejecuta 300 pruebas

aleatorias sobre arreglos de tamaño 1 a 200 con valores repetidos, comparando los resultados contra `sorted()`.

2. Complejidad del código y mantenibilidad

Este punto no se refiere a la complejidad asintótica de tiempo de ejecución (que se aborda en la sección 3), sino a la complejidad del código en sí: su tamaño, estructura y los aspectos que percibimos como fuentes potenciales de dificultad para mantenerlo. La siguiente tabla resume las métricas básicas de cada archivo:

Archivo	Líneas de código	Funciones	Recursión
src/lomuto.py	21	1	No
src/quickselect_naive.py	31	1	No (iterativa)
src/quickselect_mom.py	68	3 (2 auxiliares)	Sí (mutua)

La partición de Lomuto (`lomuto.py`, 21 líneas) es el componente más simple: una sola función, sin recursión, con un único ciclo `for` y una invariante de índices (`i`, `j`) fácil de verificar y depurar. Su mantenibilidad es alta: el código es corto, autocontenido y su lógica se puede razonar línea por línea sin conocimiento adicional del resto del sistema.

El `quickselect` con pivote ingenuo (`quickselect_naive.py`, 31 líneas) añade solo una capa de control sobre la partición de Lomuto: un ciclo `while` que decide en qué mitad continuar buscando. Se optó deliberadamente por una versión iterativa en lugar de la recursión de cola original, para evitar el límite de recursión de Python en el peor caso (arreglo ya ordenado), donde la profundidad de recursión sería $O(n)$. Esta es en sí misma una decisión que un mantenedor nuevo debe entender: el código "parece" que debería ser recursivo dado el algoritmo que implementa, y la razón de la conversión a ciclo no es obvia sin el comentario correspondiente.

El `quickselect` con mediana de medianas (`quickselect_mom.py`, 68 líneas) es más del doble de largo que la versión ingenua y, en nuestra opinión, notablemente más difícil de mantener, por varias razones concretas:

- Tiene tres funciones en lugar de una (`quickselect_mom`, `_median_of_medians` y `_partition_around_value`), con recursión mutua entre las dos primeras: `quickselect_mom` llama a `_median_of_medians` para obtener el pivote, y `_median_of_medians` llama recursivamente a `quickselect_mom` sobre el arreglo de medianas. Seguir el flujo de control exige mantener en mente dos funciones simultáneamente.
- La función `_partition_around_value` necesita reubicar el valor del pivote (que ya no es necesariamente el último elemento del segmento, como sí lo es en la versión ingenua) mediante

una búsqueda lineal antes de poder reutilizar `lomuto_partition`. Este paso adicional es una fuente plausible de bugs: si el valor del pivote aparece repetido, la búsqueda se detiene en la primera coincidencia dentro del rango, lo cual es correcto pero no es evidente a simple vista, y cualquier modificación futura a esta función debe preservar ese comportamiento con cuidado.

- El manejo de grupos de 5 elementos (división del arreglo en bloques, ordenamiento de cada bloque, extracción de su mediana) introduce aritmética de índices adicional (por ejemplo $\min(i + 5, \text{high} + 1)$ para el último grupo incompleto) que es un punto común de errores de "off-by-one".
- Al tratarse de un algoritmo con más piezas móviles, escribir pruebas unitarias exhaustivas (más allá de las pruebas de corrección global contra `sorted()`) requeriría cubrir más casos borde: grupos incompletos, muchos valores repetidos entre medianas, arreglos de tamaño menor a 5, etc.

En resumen, el costo de la garantía de $O(n)$ en el peor caso que ofrece la mediana de medianas no es solo un costo en tiempo de ejecución promedio (ver sección 4), sino también un costo de complejidad de código: más líneas, más funciones, recursión mutua y más aritmética de índices, todo lo cual incrementa la superficie de mantenimiento frente a la sencillez de la versión ingenua.

3. Análisis de los tiempos asintóticos de ejecución

3.1 Partición de Lomuto

La partición de Lomuto recorre el segmento `arr[low..high]` exactamente una vez, realizando una comparación por cada elemento (excepto el pivote). Su complejidad es, por lo tanto, $\Theta(n)$ tanto en el mejor como en el peor caso, donde $n = \text{high} - \text{low} + 1$ (Cormen et al., 2022, cap. 7).

3.2 Quickselect con pivote ingenuo

Cuando el pivote se elige siempre como el último elemento del segmento, el tamaño del subproblema se reduce, en promedio, por un factor constante en cada llamada, lo que produce una recurrencia esperada del tipo $T(n) = T(n/2) + O(n)$ que resuelve en $\Theta(n)$ (caso promedio). Sin embargo, si la entrada ya está ordenada (o inversamente ordenada), el pivote elegido siempre es el elemento extremo, produciendo la partición más desbalanceada posible en cada paso; la recurrencia se convierte en $T(n) = T(n-1) + O(n)$, cuya solución es $\Theta(n^2)$ (Cormen et al., 2022, cap. 9, "Medians and Order Statistics"). Este es precisamente el comportamiento que se observa empíricamente en la sección 4 con la entrada ya ordenada.

3.3 Quickselect con mediana de medianas

El algoritmo de mediana de medianas (Blum, Floyd, Pratt, Rivest y Tarjan, 1973) garantiza que, al dividir el arreglo en grupos de 5 y tomar la mediana de las medianas de los grupos como pivote, al menos la mitad de los grupos tienen su mediana menor o igual al pivote elegido, y la otra mitad, mayor o igual. Esto implica que, de cada grupo de 5 en al menos la mitad de los grupos, 3 elementos son $\leq$ (o $\geq$) el

pivote, lo que acota el tamaño de cada una de las dos partes resultantes de la partición a, como máximo, $7n/10 + O(1)$ elementos.

La recurrencia resultante es $T(n) \leq T(n/5) + T(7n/10) + O(n)$: el término $T(n/5)$ corresponde a encontrar recursivamente la mediana de las $n/5$ medianas de grupo, y $T(7n/10)$ corresponde a la recursión sobre la partición resultante en el peor caso. Como $1/5 + 7/10 = 9/10 < 1$, esta recurrencia resuelve en $T(n) = O(n)$ (método de sustitución / árbol de recursión), lo que garantiza tiempo lineal incluso en el peor caso, a diferencia del quickselect ingenuo (Cormen et al., 2022, cap. 9; MIT OCW 6.046J, notas de la clase 1, Devadas & Leiserson, 2012; Shor, notas del curso 18.310, MIT).

Es importante notar que esta garantía de $O(n)$ en el peor caso es asintótica y sobre el número de comparaciones en el modelo general de comparación; no dice nada sobre la constante oculta detrás de la notación O , que en la práctica es considerablemente mayor que la del quickselect ingenuo, debido al trabajo adicional de ordenar cada grupo de 5, calcular medianas y recursar dos veces por nivel (una para encontrar el pivote y otra para continuar la búsqueda). Este contraste entre la garantía teórica y el comportamiento práctico se discute con datos concretos en la sección siguiente.

4. Benchmark de las implementaciones

4.1 Metodología

Se generaron tres tipos de arreglo de entrada: (1) aleatorio, con valores tomados uniformemente al azar; (2) con muchos valores duplicados, restringidos al rango $[0, 9]$; y (3) ya ordenado (ascendente), que corresponde al peor caso teórico del quickselect con pivote ingenuo. Para cada tipo de entrada se probaron distintos tamaños n , y se midió el tiempo de ejecución (mínimo de entre 1 y 3 repeticiones, usando `time.perf_counter`) necesario para calcular la mediana ($k = n // 2$) con tres métodos: `quickselect_naive`, `quickselect_mom`, y `sorted(arr)[k]` como referencia de línea base. Los resultados completos están en `results/benchmark.csv` del repositorio, y se grafican a continuación en escala log-log, un panel por tipo de entrada.

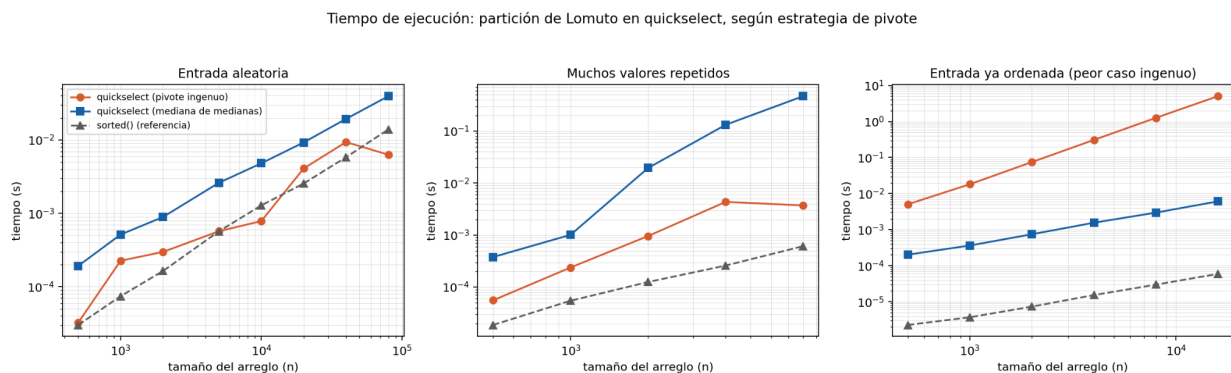


Figura 1. Tiempo de cálculo de la mediana en función de n , para los tres tipos de entrada (ejes log-log).

4.2 Discusión y contraste con lo esperado teóricamente

Entrada aleatoria: `quickselect_naive` es consistentemente más rápido que `quickselect_mom` en todos los tamaños probados, y ambos muestran un crecimiento aproximadamente lineal en n , consistente con el caso promedio $O(n)$ esperado para ambas variantes (sección 3). La diferencia entre ellas refleja la mayor constante oculta de la mediana de medianas (ordenar grupos de 5, recursión adicional). `sorted()` —Timsort implementado en C— resulta competitivo o incluso más rápido que ambas implementaciones en Python puro para n pequeño y mediano, lo cual es esperable: una constante mucho menor gracias a la implementación en C compensa, en este rango de n , la ventaja algorítmica de no ordenar todo el arreglo.

Entrada ya ordenada: aquí se observa con claridad el peor caso teórico de `quickselect_naive`. Su tiempo se degrada de forma catastrófica, de 0.005 s en $n = 500$ a 5.04 s en $n = 16\,000$, un crecimiento consistente con $O(n^2)$: al ser el pivote siempre el último elemento y la entrada estar ya ordenada, cada partición es maximalmente desbalanceada (reduce el problema en solo un elemento por paso), tal como predice el análisis de la sección 3.2. En contraste, `quickselect_mom` se mantiene rápido y estable (de 0.0002 s a 0.006 s), confirmando empíricamente su garantía teórica de $O(n)$ en el peor caso incluso sobre esta entrada adversarial. `sorted()` es extremadamente rápido en este caso gracias al comportamiento adaptativo de Timsort, que detecta runs (secuencias ya ordenadas) existentes en la entrada y los aprovecha directamente.

Entrada con muchos duplicados: en este escenario el patrón se invierte respecto a la entrada aleatoria: `quickselect_mom` se degrada fuertemente, de 0.0004 s en $n = 500$ a 0.47 s en $n = 8000$, siendo notablemente más lento que `quickselect_naive` en el mismo rango. Esto ocurre porque, con solo 10 valores distintos posibles, muchas medianas de grupo son iguales entre sí, lo que hace que la partición alrededor de un valor muy repetido no sea tan balanceada en la práctica como lo predice el análisis de peor caso genérico; además, la búsqueda lineal en `_partition_around_value` para reubicar el pivote por valor (necesaria porque, a diferencia de la versión ingenua, el pivote no está garantizado al final del segmento) añade overhead adicional cuando hay muchas coincidencias. Este resultado ilustra que la garantía asintótica de la mediana de medianas está probada sobre el número de comparaciones en el modelo general de comparación, y no captura necesariamente el comportamiento práctico frente a distribuciones con pocos valores distintos, donde el costo constante adicional del algoritmo pesa más que su ventaja teórica de peor caso.

En conjunto, el benchmark confirma cualitativamente las tres predicciones teóricas de la sección 3 (comportamiento lineal promedio de ambas variantes, degradación cuadrática del pivote ingenuo sobre entrada ordenada, y robustez de la mediana de medianas en ese mismo caso), a la vez que evidencia que, en el rango de n considerado y para datos con pocos valores distintos, la constante oculta de la mediana de medianas puede hacerla más lenta en la práctica que la alternativa ingenua e incluso que ordenar todo el arreglo.

5. Repositorio de código

El código fuente completo de la solución (backend, frontend, algoritmos, pruebas de corrección y scripts de benchmark) se mantiene en un repositorio privado de GitHub, con historial de commits de ambos integrantes del equipo:

Repositorio: <https://github.com/juanpablocarreram/ProyectoAlgoritmos>

Integrantes del equipo y su participación en el repositorio:

- Juan Pablo Carrera Martinez (juanpablo.carrera.m@gmail.com)
- Diego Sanchez Lopez (diegosan1019@gmail.com)

El repositorio se mantiene publico y se ha invitado al profesor como colaborador para que pueda revisar el historial de commits de ambos integrantes.

Referencias

Blum, M., Floyd, R. W., Pratt, V., Rivest, R. L., & Tarjan, R. E. (1973). Time bounds for selection. Journal of Computer and System Sciences, 7(4), 448–461.

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms (4a ed.), Capítulo 9: Medians and Order Statistics. MIT Press.

Devadas, S. & Leiserson, C. E. (2012). 6.046J Design and Analysis of Algorithms, notas de la clase 1: https://ocw.mit.edu/courses/6-046j-design-and-analysis-of-algorithms-spring-2012/c3a8610ea2120258eaa0d71f95f59fde/MIT6_046JS12_lec01.pdf

Shor, P. Notas del curso 18.310, MIT: The Median Algorithm: <https://math.mit.edu/~shor/18.310/median.pdf>

Sackmann, J. Datos de tenis ATP (CC BY-NC-SA 4.0): https://github.com/JeffSackmann/tennis_atp